

ISLAMIC UNIVERSITY OF TECHNOLOGY



ARTIFICIAL INTELLIGENCE LAB

CSE 4712

Lab 6

Author:

Ishmam Tashdeed
Mubtasim Kamal Dihan
CSE, IUT

Contents

1	Introduction to MiniZinc	2
1.1	Variables and Domains	2
1.2	Constraints	2
1.3	Logical Relations	2
1.4	Solving	3
1.5	Arithmetic and Logical Operators	3
1.6	Show All Solutions	4
1.7	Output in MiniZinc	4
2	Tasks	4
2.1	Task 1	4
2.2	Task 2	5
2.3	Task 3	6
2.4	Task 4	7

1 Introduction to MiniZinc

In this lab, you will be introduced to MiniZinc, a *declarative constraint modeling language* commonly used in AI and Operations Research for modeling constraint satisfaction and optimization problems (CSPs and COPs). In MiniZinc you describe the *variables*, their *domains*, and the *constraints* that must hold. The MiniZinc solver then searches for values that satisfy all constraints. We will use the MiniZinc IDE for writing and executing models. You will find the download link and a detailed tutorial of MiniZinc here: [MiniZinc Documentation](#). For quick prototyping, you can use the [Online Compiler](#).

1.1 Variables and Domains

In MiniZinc, we do not define facts like in Prolog. Instead, we define **decision variables** and specify the **domain** of values they can take.

```
1 var 1..4: animal;  
2 var 1..4: plant;
```

This example defines two variables, **animal** and **plant**, both of which can take values from 1 to 4. These numbers can represent symbolic values such as: 1 = cat, 2 = dog, 3 = mouse, 4 = butterfly.

1.2 Constraints

Constraints in MiniZinc restrict the values that variables can take. They express what must be true in any valid solution.

```
1 constraint age_max = 25;
```

This forces the solver to assign the value 25 to **age_max**. We can also use variables inside constraints:

```
1 constraint age_max = age_oscar;
```

This means Max and Oscar must have the same age.

1.3 Logical Relations

In MiniZinc, logical relationships are expressed using constraints rather than rules. For example, consider the statement: someone is happy if they like pizza.

```

1 var bool: likes_pizza_john;
2 var bool: happy_john;
3
4 constraint happy_john = likes_pizza_john;

```

This means John is happy if and only if he likes pizza. More complex logic can be expressed using logical operators:

```

1 constraint happy_john -> likes_pizza_john;

```

This reads as: if John is happy, then he must like pizza.

1.4 Solving

After defining all variables and constraints, we tell MiniZinc what kind of solution we want.

```

1 solve satisfy;

```

This asks the solver to find any solution that satisfies all constraints. MiniZinc can also handle optimization problems using: `solve minimize` or `solve maximize`.

1.5 Arithmetic and Logical Operators

MiniZinc supports standard arithmetic operators for expressing numerical relationships between variables. These operators are commonly used inside constraints to model equations and inequalities, such as addition (+), subtraction (-), multiplication (*), integer division (`div`), and modulo (`mod`). In addition, MiniZinc provides logical operators to combine or relate conditions, including conjunction (AND, `/\`), disjunction (OR, `\/`) and negation (NOT, `not`).

```

1 var 1..10: x;
2 var 1..10: y;
3
4 constraint x + y = 10;
5 constraint x * 2 <= y;
6 constraint y div 2 = x;
7 constraint y mod 2 = 0;
8
9 constraint (x + y = 10) /\ (y mod 2 = 0);
10 constraint (x * 2 <= y) \/ (y div 2 = x);

```

1.6 Show All Solutions

By default, MiniZinc displays only the first solution it finds. To see all possible solutions, click the *Show Configuration Editor* button in the top-right toolbar. In the configuration editor, go to the *Options* section and locate the *Solving* subsection. Change the selection under *Default behavior* to *User-defined behavior*, then go to the *Satisfaction problem* part and uncheck the box labeled *Stop after this many solutions*. Leaving it unchecked allows MiniZinc to enumerate all solutions. The IDE will now display every valid solution instead of only the first one.

1.7 Output in MiniZinc

In MiniZinc, the `output` keyword is used to display the values of variables after solving a model. You can include text and use the `show()` function to convert variable values into readable strings. This allows you to label and format the output clearly. For example:

```
1 var 1..10: x;  
2 var 1..10: y;  
3  
4 solve satisfy;  
5  
6 output [  
7   "x = ", show(x), "\n",  
8   "y = ", show(y), "\n"  
9 ];
```

The output section can display results for each solution, and when finding all solutions, MiniZinc will repeat the output for every valid assignment.

2 Tasks

In this lab, you are not allowed to use any built-in functions. All constraints must be expressed using only basic arithmetic, comparison, and logical operators.

2.1 Task 1

As shown in Figure 1, Sulawesi, an Indonesian island, is divided into six provinces: Gorontalo (G), West Sulawesi (WS), South Sulawesi (SS), Central

Sulawesi (CS), Southeast Sulawesi (SES), and North Sulawesi (NS).



Figure 1: A Map of Sulawesi

We want to color each province either Red (R), Yellow (Y), or Blue (B) in such a way that no two neighboring regions (provinces that share their borders) have the same color. Formulate the problem as a constraint satisfaction problem (CSP) using MiniZinc and explore the possible solution(s).

2.2 Task 2

Four friends, Z, I, F, and N came to a restaurant near their university. The restaurant serves Special Rice (S), Biryani Rice (B), Kashmiri Naan (K), and Paratha (P). You overhear their conversations, and the following preferences are known:

- Z does not like Paratha.

- I and F want to grab a bite of each other's food, so they want to order different dishes.
- F likes Rice items. So F will either take Special Rice or Biryani Rice.
- Z copies I and will order the same dish as I, but their dish must be different from the rest.
- N will not order Kashmiri Naan as they had it earlier.

We want to determine who will order which food. Formulate the problem as a constraint satisfaction problem (CSP) using MiniZinc and explore the possible solution(s).

2.3 Task 3

Four people, A, S, M, and R are looking to rent space in an apartment building. There are three floors in the building: 1, 2, and 3 (where 1 is the lowest and 3 is the highest). More than one person can live on a single floor, but each person must be assigned to some floor. The following constraints must be satisfied:

- A and S must not live on the same floor.
- If A and M live on the same floor, they must both be living on floor 2.
- If A and M live on different floors, one of them must be living on floor 3.
- R must not live on the same floor as anyone else.
- R must live on a higher floor than M.

We want to determine who will take which floor. Formulate the problem as a constraint satisfaction problem (CSP) using MiniZinc and explore the possible solution(s).

2.4 Task 4

Assume that six friends R, A, F, I, T, and S are standing in a queue, each occupying a unique spot among the six possible positions labeled 1 to 6. The following constraints must be satisfied:

- F is standing in between A and I.
- S and R are standing next to each other.
- T is either at the front of the line or at the back of the line.
- S has exactly one person behind them.

We want to determine who is standing in which position. Formulate the problem as a constraint satisfaction problem (CSP) using MiniZinc and explore the possible solution(s).